Analytics Engineer — Interview Questions & Answers

A Practical Interview Preparation Guide

Target Role

Analytics Engineer

Core Stack

SQL · dbt · Snowflake · Data Modeling · Git · Airflow · Fivetran · Tableau

---

1. What is an Analytics Engineer?

Answer

An Analytics Engineer sits between traditional data engineering and analytics.

The main responsibility is to take raw data from different sources and transform it into **clean, reliable, well-modeled datasets** that analysts and BI tools can use.

Typically, an Analytics Engineer works heavily with:

- SQL
- dbt
- Data warehouses such as Snowflake
- Data modeling
- Data quality testing
- Documentation
- Git
- BI tools

A typical flow is:

```
Data Sources
     ↓
Fivetran / ingestion
     ↓
Snowflake
     ↓
```

```
dbt
     ↓
Staging
     ↓
Intermediate
     ↓
Fact & Dimension Models
     ↓
Tableau / BI
```

Interview version

> "An Analytics Engineer focuses on transforming raw warehouse data into trusted, analytics-ready datasets. I would typically use SQL and dbt for transformation and modeling, Snowflake as the warehouse, Git for version control, and BI tools such as Tableau for consumption."

---

2. What is the difference between a Data Engineer, Analytics Engineer, and Data Analyst?

Role	Primary responsibility
Data Engineer	Build and maintain data pipelines/infrastructure
Analytics Engineer	Transform and model data for analytics
Data Analyst	Analyze data and provide business insights

Example

Suppose Salesforce contains opportunity data.

Data Engineer:

```
Salesforce
    ↓
Extract
    ↓
Load
    ↓
Snowflake
```

Analytics Engineer:

```
Raw Opportunity
       ↓
Clean
       ↓
Join Account + User
       ↓
Apply business logic
       ↓
fct_opportunity
```

Data Analyst:

```
fct_opportunity
       ↓
Analyze conversion
       ↓
Build report
       ↓
Business recommendation
```

Interview version

> "The boundaries can overlap, but generally the Data Engineer focuses on getting reliable data into the platform, the Analytics Engineer transforms and models that data for analytics, and the Data Analyst uses the curated data to answer business questions."

---

3. What is a data warehouse?

Answer

A data warehouse is a centralized system designed to store and analyze large amounts of structured data.

Examples include:

- Snowflake
- BigQuery
- Amazon Redshift
- Databricks SQL Warehouse

A warehouse is different from an operational database.

An operational database is usually optimized for applications and transactions.

A data warehouse is optimized for:

- Analytics
- Aggregations
- Reporting
- Business intelligence
- Large analytical queries

---

4. What is Snowflake?

Answer

Snowflake is a cloud-based data platform/data warehouse used to store, process, and analyze data.

One important concept is that Snowflake separates:

```
Storage
   +
Compute
```

Data is stored independently from the compute resources used to query it.

Snowflake uses virtual warehouses for compute.

For example:

```
Snowflake
│
├── Database
│    └── Schema
│         ├── Tables
│         ├── Views
│         └── Models
│
└── Virtual Warehouse
       └── Compute
```

Interview version

> "Snowflake is a cloud data warehouse and data platform. In an Analytics Engineering workflow, I would typically use Snowflake to store raw and transformed datasets and provide the compute required to execute SQL and dbt transformations."

---

5. Is Snowflake only used for storing data?

Answer

No.

This is an important distinction.

Snowflake provides both **storage and compute**.

For example, when dbt runs:

```
SELECT
    customer_id,
    SUM(revenue)
FROM orders
GROUP BY customer_id
```

the SQL is executed by Snowflake's compute resources.

So:

```
Snowflake
├── Stores data
├── Executes SQL
├── Performs transformations
├── Handles access control
└── Provides analytical compute
```

dbt doesn't replace Snowflake.

dbt generates/manages transformation logic, while Snowflake actually executes the SQL.

---

6. What is dbt?

Answer

dbt stands for **data build tool**.

dbt is primarily used to transform data inside a data warehouse using SQL.

For example:

```
Raw data
   ↓
Snowflake
   ↓
dbt
   ↓
Cleaned models
   ↓
Analytics-ready data
```

dbt helps with:

- SQL transformations
- Data modeling
- Testing
- Documentation
- Dependency management
- Lineage
- Version control
- Incremental processing
- Reusable macros

---

7. What does dbt actually do?

Suppose we write:

```sql
SELECT
    customer_id,
    SUM(amount) AS total_sales
FROM {{ ref('stg_orders') }}
GROUP BY customer_id
```

dbt:

1. Understands the dependency on `stg_orders`
2. Resolves `ref()`
3. Generates the appropriate SQL
4. Sends the SQL to the warehouse
5. Creates/updates the target model
6. Can test the resulting data
7. Tracks lineage and documentation

The actual computation happens in Snowflake.

Simple mental model

```
You write SQL
      ↓
dbt manages SQL
      ↓
Snowflake executes SQL
      ↓
Model is created
```

---

8. Why do we need dbt if Snowflake can transform data?

This is a very common interview question.

Answer

Snowflake can absolutely execute SQL transformations.

The advantage of dbt is that it provides an **engineering framework around those transformations**.

Without dbt, you might have many SQL scripts.

```
query1.sql
query2.sql
query3.sql
query4.sql
```

With dbt, you get:

```
Models
Tests
Documentation
Dependencies
Lineage
Macros
```

```
Version control
Deployment practices
```

So:

> **Snowflake provides the engine. dbt provides the transformation development framework.**

---

9. What is ELT?

ELT means:

```
Extract
Load
Transform
```

The transformation happens **after the data is loaded into the warehouse**.

Example:

```
Salesforce
    ↓
Extract
    ↓
Snowflake
    ↓
dbt
    ↓
Transform
```

This is different from traditional ETL:

```
Source
  ↓
Extract
  ↓
Transform
  ↓
Load
```

```
  ↓
Warehouse
```

Modern cloud warehouses make ELT very practical because they provide scalable compute for transformations.

---

10. What is the difference between ETL and ELT?

ETL	ELT
Extract	Extract
Transform	Load
Load	Transform
Transformation before warehouse	Transformation inside warehouse

Example

Traditional:

```
Salesforce
 ↓
ETL tool
 ↓
Transform
 ↓
Snowflake
```

Modern:

```
Salesforce
 ↓
Fivetran
 ↓
Snowflake
 ↓
dbt
```

The second approach is ELT.

---

11. What does Fivetran do?

Answer

Fivetran is primarily an **automated data integration/ingestion tool**.

It extracts data from sources and loads it into destinations such as Snowflake.

For example:

```
Salesforce
     ↓
   Fivetran
     ↓
  Snowflake
```

Fivetran handles much of the operational complexity around:

- Connecting to sources
- Extracting data
- Loading data
- Incremental synchronization
- Schema changes
- Scheduling/syncing

It is generally not where we put complex analytics business logic.

That transformation is often handled by dbt.

---

12. Why can't we directly connect Salesforce to Snowflake?

Technically, you can build a custom integration.

But the question is about **why use Fivetran or another ingestion tool**.

A direct/custom connection means you have to manage things such as:

- Authentication
- API limits
- Incremental extraction
- Changed records

- Deleted records
- Schema changes
- Failures
- Retry logic
- Scheduling
- Monitoring

Fivetran provides managed connectors that handle much of this complexity.

So:

```
Salesforce → Fivetran → Snowflake
```

is usually much easier to maintain than building the entire ingestion framework yourself.

---

13. What does Airflow do?

Answer

Airflow is primarily a **workflow orchestration tool**.

It determines:

> What should run, when should it run, and in what order?

For example:

```
Fivetran Load
      ↓
dbt Staging
      ↓
dbt Intermediate
      ↓
dbt Marts
      ↓
dbt Tests
      ↓
Tableau Refresh
```

Airflow can orchestrate these tasks and manage dependencies.

---

14. If dbt already exists, why do we need Airflow?

This is another important interview question.

dbt handles **data transformation**.

Airflow handles **workflow orchestration**.

dbt asks:

> How should data be transformed?

Airflow asks:

> When should this process run and what should happen before/after it?

For example:

```
Airflow
   │
   ├── Run ingestion
   │
   ├── Run dbt
   │
   ├── Run tests
   │
   └── Trigger downstream process
```

So:

```
Fivetran = Ingestion
Snowflake = Warehouse/Compute
dbt = Transformation
Airflow = Orchestration
Tableau = Visualization
```

---

15. What is a dbt model?

A dbt model is generally a SQL file that represents a transformation.

Example:

```
models/
└── staging/
    └── stg_orders.sql
```

Inside:

```sql
SELECT
    order_id,
    customer_id,
    order_date,
    amount
FROM {{ source('sales', 'orders') }}
```

dbt uses this SQL to create a database object such as a view or table, depending on the materialization.

---

16. What is `ref()` in dbt?

`ref()` creates a dependency between dbt models.

Example:

```sql
SELECT *
FROM {{ ref('stg_orders') }}
```

Instead of hardcoding:

```sql
FROM analytics.staging.stg_orders
```

dbt understands the model dependency.

It can therefore build:

```
stg_orders
     ↓
int_orders
     ↓
fct_sales
```

This also creates lineage.

17. What is `source()` in dbt?

`source()` references raw/source data declared in dbt.

Example:

```
SELECT *
FROM {{ source('salesforce', 'opportunity') }}
```

The source might be declared in YAML:

```
sources:
  - name: salesforce
    tables:
      - name: opportunity
```

This allows dbt to track the relationship between source data and downstream models.

---

18. What is the typical dbt project structure?

A common structure is:

```
dbt_project/
│
├── models/
│   ├── staging/
│   ├── intermediate/
│   └── marts/
│
├── macros/
├── snapshots/
├── seeds/
├── tests/
├── analyses/
├── dbt_project.yml
└── packages.yml
```

The exact structure can vary by organization.

---

19. What is a staging model?

Staging models are generally close to the raw/source data.

Typical responsibilities:

- Rename columns
- Standardize data types
- Clean obvious issues
- Normalize naming
- Basic transformations

Example:

```
raw_salesforce_opportunity
            ↓
stg_opportunity
```

A staging model should generally avoid heavy business logic.

---

20. What is an intermediate model?

Intermediate models are used for more complex transformation logic.

For example:

```
stg_orders
stg_customers
      ↓
int_orders_enriched
```

The model might:

- Join datasets
- Apply business rules
- Calculate intermediate metrics
- Resolve complex transformations

---

21. What is a mart?

A mart is an analytics-facing model designed around a business process or subject area.

Examples:

```
fct_sales
fct_orders
fct_opportunities

dim_customer
dim_product
dim_sales_rep
```

These are the models that analysts and BI tools often consume.

---

22. What is a fact table?

A fact table stores measurable business events.

Example:

```
fct_sales
----------------
order_id
customer_id
product_id
date_id
quantity
revenue
discount
```

Typical measures include:

- Revenue
- Quantity
- Cost
- Profit
- Discount

---

23. What is a dimension table?

A dimension table contains descriptive information about business entities.

Examples:

```
dim_customer
dim_product
dim_sales_rep
dim_date
```

For example:

```
dim_customer

customer_id
customer_name
city
country
segment
```

---

24. What is a star schema?

A star schema contains a central fact table connected to dimension tables.

```
             dim_customer
                  |
                  |
dim_product ── fct_sales ── dim_date
                  |
                  |
             dim_sales_rep
```

The fact table contains events/measures.

Dimensions contain descriptive attributes.

This structure is commonly used for analytics and BI.

---

25. What is grain?

This is one of the MOST important Analytics Engineer concepts.

Grain means:

> **What does one row represent?**

Example:

```
fct_sales
```

might have the grain:

> One row = one order line.

Another table could have:

> One row = one customer per month.

These are completely different grains.

Before building a model, you should clearly define its grain.

---

26. Why is grain important?

Suppose we have:

```
Orders
Order 101 → ₹1,000
Order 102 → ₹2,000
```

And customer table:

```
Customer A
Customer B
```

If we accidentally join tables with different grains, we may duplicate records.

For example:

```
Sales
Customer
Target
```

If `Target` contains one target per sales rep per month but we join it directly to order-level data, the target can be repeated across every order.

This can produce incorrect totals.

Therefore:

> **Always identify the grain before joining datasets.**

---

27. What is cardinality?

Cardinality describes the relationship between records.

Common relationships:

```
1 : 1
1 : many
many : 1
many : many
```

Example:

```
Customer → Orders
```

Usually:

```
1 customer
    ↓
many orders
```

So this is a:

```
1 : many
```

relationship.

28. What is a surrogate key?

A surrogate key is an artificial/system-generated key used to uniquely identify a record.

For example:

```
customer_sk
```

could be:

```
10001
10002
10003
```

instead of using the source's natural identifier.

Surrogate keys are particularly useful in dimensional modeling and slowly changing dimensions.

---

29. What is a natural key?

A natural key is an identifier that naturally comes from the business/source system.

Example:

```
customer_id = C10025
```

A Salesforce record ID can act as a natural/business-source identifier.

---

30. What is a slowly changing dimension?

Slowly Changing Dimensions, or SCDs, describe how changes to dimensional attributes are handled over time.

Example:

A customer changes city:

```
Before:
Customer A → Delhi

After:
Customer A → Mumbai
```

Type 1

Overwrite the old value.

```
Customer A → Mumbai
```

History is not preserved.

Type 2

Preserve history.

```
Customer A | Delhi  | 2025-01-01 | 2026-05-10
Customer A | Mumbai | 2026-05-11 | NULL
```

Type 2 is useful when historical reporting matters.

---

31. What are dbt tests?

dbt tests help validate data quality.

Common tests include:

`unique`

Checks that values are unique.

`not_null`

Checks that a field doesn't contain NULL.

`accepted_values`

Checks that values belong to an expected set.

Example:

```
stage ∈
New
Qualified
Closed Won
Closed Lost
```

`relationships`

Checks referential integrity.

For example:

```
fct_orders.customer_id
        ↓
dim_customer.customer_id
```

---

32. Why are data quality tests important?

Imagine a Tableau dashboard shows:

```
Revenue = ₹10M
```

But the underlying data contains duplicate orders.

The dashboard may look perfectly fine while showing incorrect information.

Tests allow us to catch these problems earlier.

A good Analytics Engineer doesn't just build transformations.

They build **trusted transformations**.

---

33. What is an incremental model in dbt?

An incremental model processes only new or changed records instead of rebuilding the entire table every time.

Suppose we have:

```
100 million rows
```

and only:

```
100,000 new rows
```

arrived today.

Instead of processing all 100 million rows, an incremental model can process the relevant new/changed data.

This can significantly improve performance and reduce compute costs.

---

34. When would you use a view vs table vs incremental model?

View

Useful when:

- Data is relatively small
- Transformation is inexpensive
- You don't need physically stored results

Table

Useful when:

- Query performance matters
- Transformation is expensive
- Data doesn't need to be rebuilt constantly

Incremental

Useful when:

- Dataset is large
- New/changed data can be identified
- Full refresh is expensive

---

35. What is a dbt snapshot?

A snapshot is used to track changes to records over time.

For example:

```
Customer status

2026-01-01 → Active
2026-05-01 → Inactive
```

Snapshots are particularly useful when you need historical versions of mutable source records.

---

36. What is a dbt macro?

A macro is reusable logic written using Jinja.

Instead of repeating the same SQL logic in many models, you can create a macro.

Conceptually:

```
Macro
   ↓
Reusable SQL logic
   ↓
Multiple models
```

This helps reduce duplication and improve maintainability.

---

37. What is Jinja in dbt?

Jinja is a templating language used by dbt to make SQL dynamic and reusable.

For example:

```
{{ ref('stg_orders') }}
```

or:

```
{% if target.name == 'prod' %}
...
{% endif %}
```

Jinja allows us to add programming-like capabilities around SQL.

---

38. What is Git and why does an Analytics Engineer need it?

Git is a version control system.

It allows developers to:

- Track changes
- Create branches
- Collaborate
- Review code
- Revert changes
- Merge work

A typical workflow:

```
main
 │
 └── feature/new-sales-model
          ↓
       Commit
          ↓
       Pull Request
          ↓
       Code Review
          ↓
       Merge
```

For dbt projects, Git is extremely important because transformation logic is code.

---

39. What is CI/CD in dbt?

CI/CD means Continuous Integration and Continuous Deployment.

For example, when someone creates a pull request:

```
Developer
    ↓
Git branch
    ↓
Pull Request
    ↓
CI pipeline
    ↓
dbt build / tests
    ↓
Code review
    ↓
Merge
```

This helps prevent broken transformations from reaching production.

---

40. How would you troubleshoot a Tableau number that doesn't match the source?

I would not immediately change the Tableau calculation.

I'd investigate systematically.

Step 1 — Check the Tableau calculation

Make sure the aggregation is correct:

```
SUM
AVG
COUNT
COUNTD
```

Step 2 — Check filters

Look for:

- Context filters
- Dimension filters
- Date filters
- Data source filters

Step 3 — Check joins/relationships

Incorrect relationships can duplicate or remove records.

Step 4 — Check grain

Ask:

> What does one row represent in each dataset?

Step 5 — Validate the dbt model

Run SQL directly against Snowflake.

Step 6 — Compare row counts

```
SELECT COUNT(*)
FROM model;
```

Step 7 — Check duplicates

```
SELECT
    order_id,
    COUNT(*)
FROM model
GROUP BY order_id
HAVING COUNT(*) > 1;
```

Step 8 — Trace backwards

```
Tableau
   ↓
Mart
   ↓
Intermediate
   ↓
Staging
   ↓
Raw
```

This approach is much better than randomly changing Tableau calculations.

---

41. How would you handle duplicate records?

First, I would understand **why the duplicates exist**.

I wouldn't automatically use `DISTINCT`.

For example:

```
SELECT *
FROM orders
```

If duplicates exist, possible causes include:

- Duplicate source records
- Many-to-many joins
- Incorrect join condition
- Multiple versions of a record
- Incremental loading issue

If the business definition says one record should exist per `order_id`, I might use:

```
ROW_NUMBER() OVER (
    PARTITION BY order_id
    ORDER BY updated_at DESC
)
```

Then retain:

```
row_number = 1
```

But the correct solution depends on the reason for duplication.

---

42. Why shouldn't we blindly use DISTINCT?

Because `DISTINCT` can hide a data problem.

Suppose:

```
Order ID | Amount
1001     | 1000
1001     | 1200
```

These are not exact duplicates.

`DISTINCT` won't solve the underlying issue.

We need to determine which record is correct.

A better approach might be:

```
ROW_NUMBER() OVER (
    PARTITION BY order_id
    ORDER BY updated_at DESC
)
```

and select the latest record.

---

43. What is a many-to-many join problem?

Suppose:

```
Sales
Rep A → 10 orders

Targets
Rep A → 3 monthly target records
```

If we join incorrectly:

```
10 sales rows
×
3 target rows
```

we can produce:

```
30 rows
```

instead of 10.

This causes inflated measures.

This is why **grain and cardinality must be understood before joining datasets.**

---

44. How would you handle NULL values?

It depends on the business meaning.

For example:

```
COALESCE(revenue, 0)
```

can replace NULL revenue with zero.

But I wouldn't automatically convert every NULL to zero.

NULL can mean:

- Unknown
- Missing
- Not applicable
- Not yet available

For example:

```
discount = NULL
```

does not necessarily mean:

```
discount = 0
```

The business meaning should be understood first.

---

45. What is data lineage?

Data lineage describes where data comes from and how it flows through transformations.

Example:

```
Salesforce
    ↓
Fivetran
    ↓
raw_opportunity
    ↓
stg_opportunity
    ↓
int_opportunity
    ↓
fct_opportunity
    ↓
Tableau
```

Lineage helps with:

- Troubleshooting
- Impact analysis
- Documentation
- Governance
- Understanding dependencies

dbt can automatically provide model-level lineage.

---

46. How would you design a dbt project?

I would generally separate models into layers.

```
models/
│
├── staging/
│   ├── stg_customer.sql
│   ├── stg_orders.sql
│   └── stg_products.sql
│
├── intermediate/
│   ├── int_orders_enriched.sql
│   └── int_customer_orders.sql
│
└── marts/
    ├── fct_sales.sql
```

```
    ├── dim_customer.sql
    ├── dim_product.sql
    └── dim_date.sql
```

The goal is to separate:

```
Source cleanup
      ↓
Transformation logic
      ↓
Business-facing models
```

This improves maintainability.

---

47. How would you optimize a slow dbt model?

I would first identify where the time is being spent.

Possible areas:

1. Reduce unnecessary data

Don't process columns or rows that aren't needed.

2. Filter earlier

Push filters closer to the source where appropriate.

3. Review joins

Look for:

- Many-to-many joins
- Large intermediate datasets
- Missing join conditions

4. Check materialization

A model might be better as a table or incremental model instead of a view.

5. Check Snowflake compute

The warehouse size and workload may need investigation.

6. Review query profile

Use Snowflake's query information/profile to identify expensive operations.

---

48. What is the difference between a view and a table?

View

A view stores the SQL definition rather than a separately materialized copy of the result.

```
Query view
    ↓
Underlying data is queried
```

Table

The transformed results are physically stored.

```
Transformation
      ↓
Stored table
```

The choice depends on:

- Query frequency
- Transformation cost
- Data volume
- Performance requirements

---

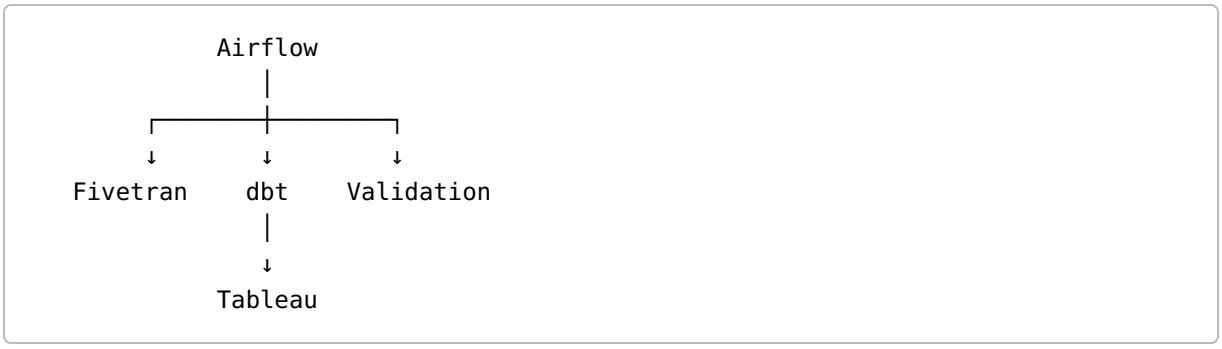
49. How does Airflow fit into a modern data stack?

A typical architecture could be:

```
Salesforce
    ↓
Fivetran
    ↓
Snowflake
```

```
    ↓
dbt
    ↓
Analytics Marts
    ↓
Tableau
```

Airflow can orchestrate the workflow:



Airflow is not necessarily responsible for doing the transformation itself.

---

50. Explain the complete project architecture.

Strong interview answer

> "In a typical project, operational data comes from sources such as Salesforce. A tool like Fivetran extracts and loads that data into Snowflake. In Snowflake, the raw data is stored in a raw layer. I then use dbt to build staging models for standardization, intermediate models for more complex transformations, and fact and dimension models for analytics. I add dbt tests such as not-null, unique, accepted-values, and relationship tests to ensure data quality. Git is used for version control and CI/CD can run dbt tests before deployment. Airflow can orchestrate the overall workflow, and Tableau can consume the final analytics-ready models."

---

51. What happens when a dbt model fails?

I would first identify:

```
Which model failed?
        ↓
What error occurred?
        ↓
Is it SQL?
```

```
        ↓
Is it source data?
        ↓
Is it dependency?
        ↓
Is it permissions?
        ↓
Is it warehouse/resource related?
```

For example, if:

```
stg_orders
```

fails, downstream models such as:

```
int_orders
fct_sales
```

may also fail because of dependencies.

I would inspect the dbt logs and error message, reproduce the issue where possible, fix the root cause, and rerun the relevant models/tests.

---

52. What is the difference between a source table and a dbt model?

Source

Data that comes from an external system.

```
Salesforce
    ↓
raw.opportunity
```

dbt model

A transformation managed by dbt.

```
raw.opportunity
       ↓
stg_opportunity
```

So:

```
Source = input
Model = transformation
```

---

53. How would you handle a source schema change?

For example, Salesforce changes:

```
customer_name
```

to:

```
customer_full_name
```

I would first determine:

1. What changed?
2. Is it expected?
3. Which downstream models depend on the field?
4. Which dashboards are affected?

Then update the relevant staging model and downstream logic.

I would run:

```
dbt build
```

or the appropriate targeted models/tests and validate downstream outputs.

This is where lineage and tests become very valuable.

---

54. What is a data contract?

A data contract defines expectations about data between producers and consumers.

It can specify things such as:

```
Column names
Data types
Required fields
Allowed values
Expected grain
Schema
```

For example:

```
order_id → required
amount → numeric
order_date → date
```

Contracts reduce unexpected downstream breakage.

---

55. What is the difference between a metric and a dimension?

A **metric/measure** is something we calculate or aggregate.

Examples:

```
Revenue
Orders
Profit
Quantity
```

A **dimension** describes how we slice the metric.

Examples:

```
Region
Product
```

```
Customer
Sales Rep
Month
```

For example:

```
Revenue by Sales Rep
```

Here:

```
Revenue = measure
Sales Rep = dimension
```

---

56. What is a semantic layer?

A semantic layer defines business metrics and relationships in a consistent way so different consumers use the same definitions.

For example:

Instead of every analyst defining:

```
Revenue
```

differently, the organization can define one trusted metric.

```
Revenue =
SUM(order_amount)
WHERE order_status = 'Completed'
```

This reduces metric inconsistency across dashboards and teams.

---

57. How does Tableau fit into Analytics Engineering?

Tableau is generally the **consumption/BI layer**.

The Analytics Engineer should ideally provide Tableau with clean, trusted models.

Instead of putting complex business logic everywhere inside Tableau:

```
Raw data
   ↓
Tableau
   ↓
50 calculations
```

we can move reusable business logic upstream:

```
Raw data
   ↓
Snowflake
   ↓
dbt
   ↓
fct_sales
   ↓
Tableau
```

This gives us:

- Reusable logic
- Better governance
- Easier testing
- Better consistency
- Simpler dashboards

---

58. What SQL topics should an Analytics Engineer know?

At minimum:

Basic SQL

```
SELECT
WHERE
GROUP BY
```

```
HAVING
ORDER BY
```

Joins

```
INNER
LEFT
RIGHT
FULL
```

Intermediate

```
CASE
COALESCE
CTEs
Subqueries
UNION
UNION ALL
```

Advanced

```
ROW_NUMBER
RANK
DENSE_RANK
LAG
LEAD
SUM OVER
AVG OVER
```

Dates

```
DATE_TRUNC
DATEADD
DATEDIFF
```

Analytics problems

```
Deduplication
Top N
Running totals
Rolling averages
MoM
```

```
YoY
Retention
Latest record
First/last event
```

---

59. SQL Interview Example — Find the latest order for every customer

```sql
WITH ranked_orders AS (

    SELECT
        customer_id,
        order_id,
        order_date,
        ROW_NUMBER() OVER (
            PARTITION BY customer_id
            ORDER BY order_date DESC
        ) AS rn

    FROM orders

)

SELECT
    customer_id,
    order_id,
    order_date
FROM ranked_orders
WHERE rn = 1;
```

Explanation

We partition by customer:

```sql
PARTITION BY customer_id
```

Then rank each customer's orders from newest to oldest:

```sql
ORDER BY order_date DESC
```

Finally:

```
WHERE rn = 1
```

returns the latest order.

---

60. SQL Interview Example — Running revenue

```
SELECT
    order_date,
    revenue,

    SUM(revenue) OVER (
        ORDER BY order_date
    ) AS running_revenue

FROM sales;
```

The window function calculates cumulative revenue without collapsing the individual rows.

---

61. SQL Interview Example — Top 3 sales reps per region

```
WITH ranked_reps AS (

    SELECT
        region,
        sales_rep,
        revenue,

        RANK() OVER (
            PARTITION BY region
            ORDER BY revenue DESC
        ) AS rnk

    FROM sales_rep_revenue

)
```

```sql
SELECT *
FROM ranked_reps
WHERE rnk <= 3;
```

This is a classic window-function problem.

---

62. What would you do if a business user says the dashboard is wrong?

I would avoid immediately assuming Tableau is the problem.

I would trace the data flow:

```
Dashboard
   ↓
Tableau calculation
   ↓
Data source
   ↓
dbt mart
   ↓
Intermediate model
   ↓
Staging model
   ↓
Raw source
```

At each stage I would compare:

- Row count
- Aggregation
- Filters
- Grain
- Duplicates
- Join behavior
- Business logic

The goal is to identify the **first point where the number becomes incorrect**.

---

63. What is the most important concept when joining data?

Grain.

Before joining:

```
Table A
```

and:

```
Table B
```

I would ask:

> What does one row represent in A?
>
> What does one row represent in B?
>
> What is the expected relationship?

For example:

```
Orders
1 row = 1 order

Targets
1 row = 1 sales rep + month
```

Joining them directly on:

```
sales_rep
```

could create duplicates.

The join might need:

```
sales_rep
+
month
```

depending on the business requirement.

---

64. What makes a good Analytics Engineer?

A strong Analytics Engineer doesn't just know SQL.

They understand the entire data lifecycle:

```
Source
  ↓
Ingestion
  ↓
Warehouse
  ↓
Transformation
  ↓
Modeling
  ↓
Testing
  ↓
Documentation
  ↓
BI
  ↓
Business
```

They also understand:

- Data quality
- Grain
- Business definitions
- Maintainability
- Performance
- Version control
- Deployment
- Debugging

---

65. Your 30-Day Interview Priority

For your transition, I would prioritize the topics in this order:

Tier 1 — Must Master

```
⭐⭐⭐⭐⭐ SQL
⭐⭐⭐⭐⭐ Data Modeling
⭐⭐⭐⭐⭐ dbt
⭐⭐⭐⭐⭐ Grain & Joins
⭐⭐⭐⭐⭐ Data Quality
```

Tier 2 — Strong Understanding

```
⭐⭐⭐⭐ Snowflake
⭐⭐⭐⭐ Git
⭐⭐⭐⭐ ELT
⭐⭐⭐⭐ Analytics Architecture
```

Tier 3 — Interview-Level Knowledge

```
⭐⭐⭐ Airflow
⭐⭐⭐ Fivetran
⭐⭐⭐ CI/CD
⭐⭐⭐ SCD
⭐⭐⭐ Incremental Models
```

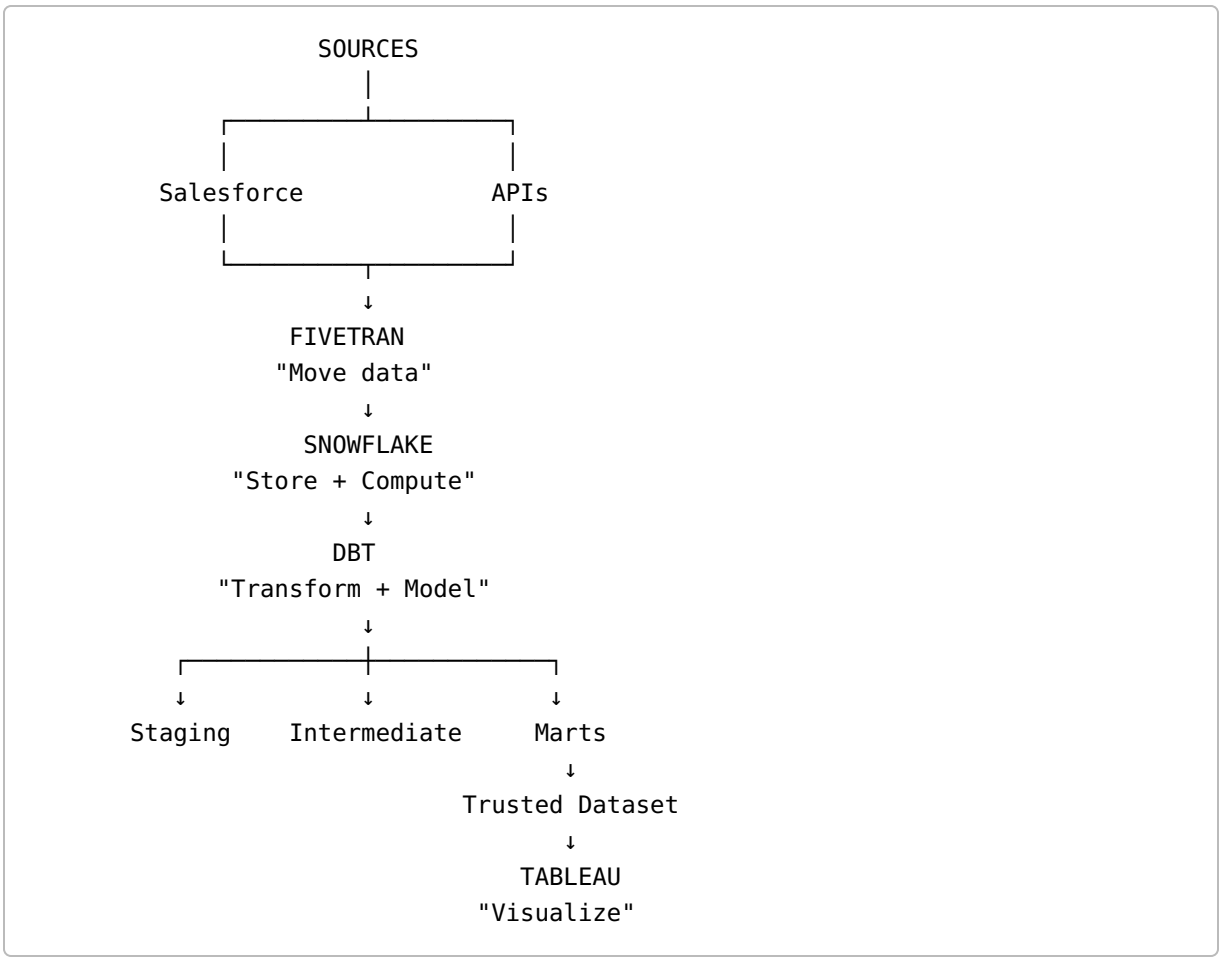
Tier 4 — Don't Over-invest Right Now

```
⭐ Spark
⭐ Kafka
⭐ Kubernetes
⭐ Advanced AWS
⭐ Advanced Python
```

---

66. The Mental Model You Should Memorize

When you see a modern Analytics Engineering stack, think:



And:

```
AIRFLOW
"Orchestrate the workflow"
```

---

67. The One-Sentence Version of Each Tool

If an interviewer asks you to explain the stack quickly:

Snowflake

> **"Cloud data warehouse/platform where we store and process analytical data."**

Fivetran

> **"Managed data ingestion tool that moves data from operational sources into the warehouse."**

dbt

> **"Transformation and analytics engineering framework that lets us build, test, document, and manage SQL models inside the warehouse."**

Airflow

> **"Workflow orchestration platform used to schedule and coordinate data processes and their dependencies."**

Git

> **"Version control system used to manage and collaborate on transformation code."**

Tableau

> **"BI and visualization platform used to consume trusted analytical datasets and deliver insights to business users."**

---

68. The Ideal End-to-End Interview Answer

If the interviewer asks:

> **"Walk me through how you would build an analytics pipeline."**

A strong answer would be:

> "I would first identify the source systems and understand the business requirements and grain of the required datasets. Data could be extracted from systems such as Salesforce using a managed ingestion tool like Fivetran and loaded into Snowflake. I would then create dbt staging models to clean and standardize the raw data, followed by intermediate models for reusable transformation logic. Finally, I would create fact and dimension models at clearly defined grains for analytics consumption. I would add dbt tests for uniqueness, nullability, accepted values, and relationships, and document the models and lineage. Git would be used for version control and CI/CD could validate changes before deployment. Airflow could orchestrate ingestion, dbt runs, tests, and downstream processes. Tableau would then consume the curated marts rather than relying on raw source data."

That is the kind of answer I want you to eventually deliver **without memorizing it word-for-word**.

Final Interview Principle

For almost every Analytics Engineer question, think through these five things:

```
1. SOURCE
   Where did the data come from?

2. GRAIN
   What does one row represent?

3. TRANSFORMATION
   What business logic is applied?

4. QUALITY
   How do we know the data is correct?

5. CONSUMPTION
   Who/what uses the final data?
```

If you develop this way of thinking, you stop sounding like someone who has simply learned **dbt commands** and start sounding like an **Analytics Engineer**.